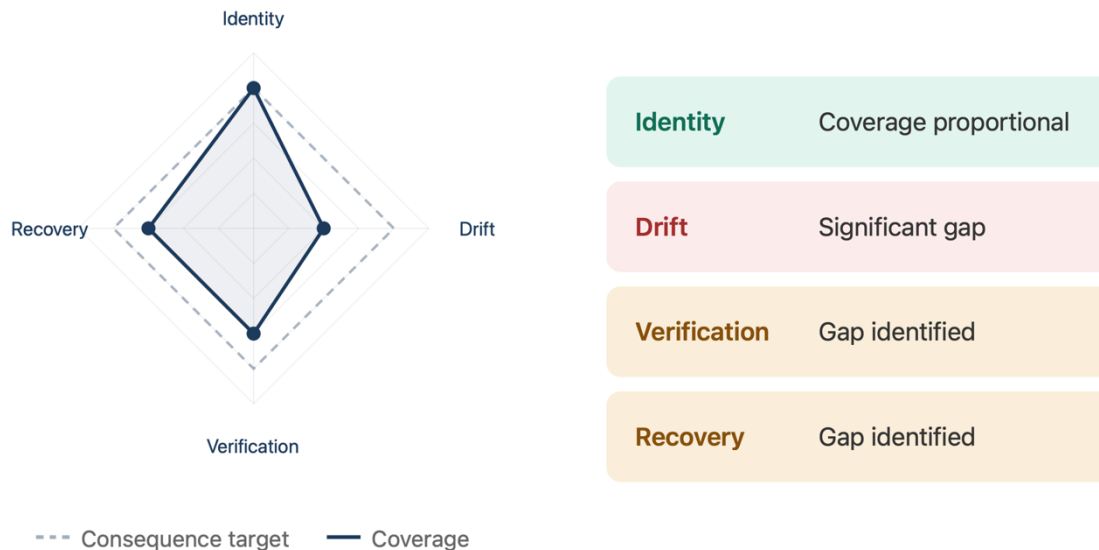


AI Governance Begins with Quantified Exposure

Using a framework grounded in established practices from quantitative finance, pharmaceutical manufacturing, and avionics safety engineering.



Most organizations today use language models somewhere in their operations. Customer support, compliance screening, document processing, financial approvals – the adoption is broad and accelerating.

Governance hasn't kept pace. Traditional software is deterministic: same input, same output. We know how to test and debug it. Language models are probabilistic – their behavior can change between calls and when providers update what sits behind their API. And the model input space is too vast for any test suite to cover meaningfully.

And unlike a model that hands you a number you know to question, a language model's output reads like an expert wrote it – fluent, confident, formatted like an authoritative answer. It makes it easy to forget you're looking at a probabilistic suggestion, not a verified fact.

This creates a coverage problem. Traditional IT governance doesn't cover shifting sand – many organizations don't have systematic visibility into how language models are deployed and operate, and reliable mechanisms to verify outputs and recover from failures.

The Consequence Question

Before evaluating coverage, you need to know whether it matters. On a scale of 1 to 5, how much does it matter if this model produces a wrong output?

If the answer is 1, wrong outputs would have little practical effect. At 5, you might be looking at regulatory action or legal liability.

If the answer is 1, the model deployment may not need formal diligence. But most production deployments score higher than people initially assume, especially when they think carefully about downstream consequences.

Four Coverage Dimensions

For any model where the consequence level warrants attention, there are four areas that require coverage. They're independent – strength in one doesn't compensate for uncertainty in another.

Identity

Do you know exactly which model and version is running the process? Not the provider or the product family – the specific model and version in your implementation. Many organizations discover that they're calling a wrapper API or framework default that tracks whatever is the current version, and don't actually know what's behind the endpoint.

Drift

If the model or its behavior changed, would you know? Providers update models behind stable API endpoints, sometimes with notice and sometimes without. If your detection mechanism is "someone would probably notice if outputs seemed different," that's not a systematic capability.

Verification

Is output quality checked before the output reaches its end use? A human reviewer, an automated quality gate, a rule-based filter – something that evaluates the work before it reaches its consequence. Verification can be expensive and consequently minimized: it's worth understanding what constitutes genuine review.

Recovery

If a bad output got through everything above, how quickly would it be caught? For some processes, the answer is minutes. For others, it might be months. The gap between when damage starts accumulating and when someone notices is the number that matters most.

Why the Proportionality Matters

These questions produce different answers at different consequence levels, and that's by design. A meeting summarizer with no drift detection is probably fine – the stakes don't justify the investment. A model that auto-approves invoices for payment with no drift detection is a different situation entirely.

The same coverage gap can be acceptable or urgent depending on what's at stake. The distance between consequence and coverage is your exposure. A framework that imposes the same rigor on every AI deployment might be asking too much in some cases.

What We're Finding

We developed our Language Model Diligence framework grounded in established practices from quantitative finance (where probabilistic model risk has been governed for decades), pharmaceutical manufacturing (where process observation replaced output testing), and avionics safety engineering (where governance rigor scales with failure consequences).

It's early days, but as we apply the five questions across industries the pattern is consistent. Most teams know what tasks their models perform. Many but not all know exactly which model and version is running. Fewer have systematic drift detection. And very few have measured how long it would take to notice if a model started producing subtly wrong outputs.

The gaps aren't random – they follow a predictable shape – and they're addressable once they're surfaced.